RuleGuard: Business Rule Conflict Detection and Resolution System

Software Requirements and Architecture Document

Akhil Sriramaneni

FA26 - COMP-699-A - PROFESSIONAL SEMINAR

Professor: David Pitts

Rivier University

1. Problem Definition and Project Scope

Organizations use business rules to make decisions about approvals, eligibility, pricing, and internal operations. When different departments create or update rules independently, the rules may overlap, contradict each other, or become impossible to satisfy. These problems can result in incorrect decisions and additional manual work.

RuleGuard will be a Python-based application that helps users identify these problems before rules are used in business processes. Its main users will be business analysts, policy administrators, developers, and testers.

The system will allow users to create and manage structured rules, analyze conflicts, understand affected conditions, generate test cases, and track issue resolution. The project will use Python, Streamlit, and SQLite and will run locally.

The initial scope includes rule validation, evaluation, overlap detection, contradictory decision detection, unreachable-rule analysis, test-case generation, and issue tracking. Production integrations, cloud deployment, enterprise authentication, and automatic policy correction are outside the initial scope.

2. Users and Stakeholders

Business analysts will create and review rules and investigate conflicts. Policy administrators will maintain rule collections and track issues. Developers will implement and maintain the analysis algorithms and application components. Testers will verify rule outcomes and system behavior. Managers will review reports and decide which policy issues require attention.

The project developer will implement, test, document, and demonstrate the application. The initial version will support a single local user environment without role-based access or multi-user collaboration.

3. Functional Requirements

Rule Management

FR-01: The system shall allow users to create rules with unique identifiers, names, conditions, and expected decisions.

FR-02: The system shall allow users to edit existing rules.

FR-03: The system shall allow users to delete rules after confirmation.

FR-04: The system shall display all saved rules.

FR-05: The system shall allow users to search and filter rules.

FR-06: The system shall organize rules into named rule sets.

FR-07: The system shall import rules from supported JSON files.

FR-08: The system shall export rules to JSON files.

Validation and Evaluation

FR-09: The system shall check required rule fields before saving.

FR-10: The system shall support equality, inequality, and numerical comparison operators.

FR-11: The system shall support AND and OR conditions.

FR-12: The system shall display errors for invalid rules and unsupported operators.

FR-13: The system shall evaluate a rule against supplied input values.

FR-14: The system shall evaluate all applicable rules in a selected rule set.

FR-15: The system shall display matched rules and their decisions.

Conflict Detection

FR-16: The system shall identify overlapping rule conditions.

FR-17: The system shall detect overlapping rules that produce different decisions.

FR-18: The system shall identify rules with impossible conditions.

FR-19: The system shall detect unreachable rules when priority and condition analysis establish that they cannot be applied.

FR-20: The system shall distinguish between overlapping rules with matching and conflicting decisions.

FR-21: The system shall identify duplicate rules.

FR-22: The system shall display the identifiers of affected rules.

FR-23: The system shall explain the conditions responsible for each confirmed conflict.

FR-24: The system shall flag cases that cannot be conclusively analyzed.

FR-25: The system shall generate conflict reports.

Testing and Resolution

FR-26: The system shall generate representative test cases for detected conflicts.

FR-27: The system shall allow users to enter custom test inputs.

FR-28: The system shall display the results of each test case.

FR-29: The system shall allow users to mark issues as open, under review, or resolved.

FR-30: The system shall allow users to add resolution notes.

FR-31: The system shall save issue statuses and resolution notes.

FR-32: The system shall allow users to rerun analysis after rule changes.

FR-33: The system shall compare issues across analysis runs.

FR-34: The system shall retain the rule version or snapshot associated with each analysis.

Interface and Storage

FR-35: The system shall provide a Streamlit interface for rule management, analysis, and issue review.

FR-36: The system shall store rules, rule sets, analysis runs, and issues in SQLite.

FR-37: The system shall restore saved records when the application restarts.

FR-38: The system shall display summary counts of rules, conflicts, unreachable rules, and resolved issues.

FR-39: The system shall display clear success and error messages.

FR-40: The system shall provide sample rules for demonstrating its main features.

4. Nonfunctional Requirements

NFR-01 — Performance: The system shall analyze 500 representative rules within five seconds under normal local conditions.

NFR-02 — Usability: A first-time user shall complete a basic rule-analysis workflow within ten minutes using the instructions.

NFR-03 — Reliability: At least 99 of 100 valid analysis runs shall complete without unexpected failures during testing.

NFR-04 — Data Integrity: The system shall validate required values and use transactions for related database updates.

NFR-05 — Maintainability: The application shall separate interface, processing, database, and testing components.

NFR-06 — Testability: Core rule-processing functions shall be testable independently of the interface.

NFR-07 — Portability: The application shall run on supported Python environments on Windows, macOS, and Linux.

NFR-08 — Compatibility: The application shall use compatible versions of Python, Streamlit, and SQLite.

NFR-09 — Security: Database operations involving user input shall use parameterized SQL queries.

NFR-10 — Privacy: The initial version shall store and process data locally without external services.

NFR-11 — Input Validation: Invalid inputs shall produce understandable errors without crashing the application.

NFR-12 — Explainability: Confirmed conflicts shall identify affected rules and supporting conditions.

NFR-13 — Accuracy: The system shall correctly classify at least 100 manually verified test cases within its supported rule model.

NFR-14 — Consistency: Identical inputs and rules shall produce identical results under the same application settings.

NFR-15 — Recoverability: Saved data shall survive normal restarts, and documented backup recovery shall be supported.

NFR-16 — Accessibility: The interface shall use readable labels, consistent navigation, and clear messages.

NFR-17 — Scalability: The initial design shall support rule sets containing at least 500 rules.

NFR-18 — Resource Usage: The application shall run on a typical laptop with 8 GB RAM without a GPU.

NFR-19 — Documentation: The repository shall include setup, dependency, and usage instructions.

NFR-20 — Reproducibility: The project shall define dependencies and provide repeatable automated tests.

5. System Architecture

RuleGuard will use three main components. The Streamlit interface will collect user input and display results. The Python processing component will validate and evaluate rules, detect conflicts, and generate explanations. The SQLite data component will save and retrieve rules, analysis results, and issue records.

The user will select a rule set, run an analysis, and review the results. The processing component will analyze the rules and send its findings to the interface. The results will be stored locally so users can review previous analyses after restarting the application.

The application will use deterministic algorithms instead of external AI services. Keeping processing logic separate from the interface will make testing and maintenance easier.

6. Data Design

SQLite will contain five main entities.

- **RuleSet:** Stores the rule-set identifier, name, and description.
- **Rule:** Stores the rule identifier, rule-set identifier, conditions, expected decision, priority, and status.
- **AnalysisRun:** Stores the analysis identifier, date, rule snapshot, and summary.
- **Issue:** Stores the issue type, affected rules, explanation, status, and resolution notes.
- **TestCase:** Stores test inputs, matched rules, expected decisions, and results.

Each rule belongs to a rule set. An analysis run belongs to a rule set and can produce multiple issues and test cases. Foreign keys and database transactions will help maintain data integrity. JSON will represent structured rule conditions. The project will use locally created sample data rather than external datasets.

7. Interfaces and APIs

The application will provide a Rule Management screen, Analysis screen, Results screen, Testing screen, Issue Review screen, and Summary Dashboard.

Users will create or import rules, select a rule set, run analysis, inspect explanations, execute test cases, and update issue statuses. Internal Python functions will handle validation, evaluation, conflict detection, test generation, and database operations. JSON import and export will support transferring rule data. A public REST API is not required for the initial version.

8. Technology Stack and Design Decisions

Python will implement the processing logic because it supports readable code and automated testing. Streamlit will provide the interactive interface without requiring a separate frontend. SQLite will provide local storage without a database server. JSON will represent rule conditions and support file exchange. pytest will support automated testing, while Git and GitHub will track source code and project progress.

PostgreSQL and a separate REST backend were considered but would add unnecessary complexity for a single-user local application. Cloud hosting and external services are also outside the initial scope.

9. Technical Proof of Concept

The proof of concept will demonstrate rule validation, condition evaluation, conflict detection, and SQLite storage. A small Streamlit interface will allow users to submit sample rules and view analysis results.

For example, two rules with overlapping age ranges but different decisions should produce a conflict finding. A rule with contradictory conditions should be identified as impossible to satisfy. The prototype will also demonstrate saving and retrieving rules.

Evidence will include screenshots, sample inputs and outputs, and automated test results collected after execution. The remaining work will include broader testing, historical comparisons, issue tracking, error handling, and performance evaluation.

10. Testing and Evaluation Plan

Unit tests will cover condition evaluation, input validation, overlaps, contradictions, unreachable rules, and test-case generation. Integration tests will verify communication between the interface, processing logic, and database. System tests will cover complete user workflows, including rule creation, analysis, issue resolution, and rerunning analysis.

At least 100 manually verified test cases will be used to evaluate analysis accuracy. Performance testing will measure whether 500 rules can be analyzed within five seconds. Usability testing will measure whether new users can complete the main workflow within ten minutes.

The results will be recorded using automated test reports, screenshots, execution times, and a requirements checklist. Actual results will be documented after the tests are executed.

11. Risks and Technical Challenges

Algorithm complexity: The system will initially support a defined set of operators and logical conditions.

Incorrect findings: Boundary cases and overlapping conditions will be tested using manually verified examples.

Unreachable-rule analysis: Definitive results will be limited to cases that the supported condition model can establish.

Data consistency: Analysis results will retain the associated rule snapshot or version.

Time constraints: Core conflict detection and rule evaluation will take priority over optional features.

Dependency issues: Compatible library versions will be documented and tested in a clean environment.

12. Development Schedule

October 17–23: Finalize requirements, architecture, and data design.

October 24–30: Build the technical proof of concept.

October 31–November 8: Implement rule management and core detection algorithms.

November 9–15: Integrate the processing logic, database, and interface.

November 16–20: Complete the minimum viable product.

November 21–27: Conduct functional, accuracy, integration, and performance testing.

November 28–December 2: Fix defects and improve usability.

December 3–5: Complete the README and technical documentation.

December 6–7: Perform final verification and prepare the demonstration.

December 8: Submit and demonstrate the final system.

References

Python Software Foundation. (n.d.). *Python documentation*. https://docs.python.org/3/

Streamlit. (n.d.). *Streamlit documentation*. https://docs.streamlit.io/

SQLite. (n.d.). *SQLite documentation*. https://www.sqlite.org/docs.html

pytest. (n.d.). *pytest documentation*. https://docs.pytest.org/en/stable/

Git. (n.d.). *Git documentation*. https://git-scm.com/doc